



EEE3027
Embedded Systems
Coursework Report
Semester 2

Submitted by c3031460

Table of Contents

General Information:	3
Part 1- Implement a shared FIFO Buffer:	3
Part 2- Implement a concurrent ML pipeline with integration of the shared FIFO memory buffer:	3
Part 3 - Asynchronous Communication Mechanism (ACM) Implementation and Performance Analysis:	5
References	7

General Information:

*“This semester's coursework involves creating a software-based embedded system using **Python**, where you will implement an **optimal first-in first-out (FIFO) buffer with shared memory** capabilities (via a **mutual exclusion (MUTEX) protocol**), leading to a **real-time forecasting application (with parallel processes)** using **machine learning (ML) models**.”*

All tasks were completed and the implementation was developed in VS Code using Python and was run on a GPU.

Part 1- Implement a shared FIFO Buffer:

The file ‘Part_1.py’ is the complete implementation of tasks 1a and 1b. The class ‘Buffer’ is the FIFO Buffer with two functions: ‘consume’ and ‘produce’ and takes the size of the buffer as a variable to adjust the size as needed and achieve a dynamic design. A mutex lock protocol has been applied for each function, so no race conditions or data trampling occur. The buffer is implemented as a circular buffer that moves through the slots until it finds one available to produce and consume.

The function ‘produce’ takes the data as variables and adds them to the buffer. Before adding them, the ‘empty’ semaphore checks for available slots in the buffer. If there is an available slot, the lock mutex is acquired, and the slot is released after the data has been written to the slot in the buffer.

The function ‘consume’ retrieves data from the buffer and stores it in a variable for use by other classes in the pipeline. The same principle applies to the consumer function; the ‘full’ semaphore is acquired to check for available data. It then locks the available slot and retrieves the data from the buffer. The mutex is then released.

Part 2- Implement a concurrent ML pipeline with integration of the shared FIFO memory buffer:

The file ‘Part_2.py’ refactors the given ML pipeline from the file ‘main_script. ipynb’ using ‘threading. Thread’ and is successfully decoupled into the following classes to enable concurrent implementation:

- Load: loads the weather data from 'weather.csv' and applies 'feature engineering' to be read by the model. This includes converting the date from an integer to a datetime object, dropping or setting 'None' values to missing data.
- Split: splits the data into training and testing sets for the model
- Preprocess: scales the data to avoid data leakage
- Batch: batches the data into frames based on the number of past input data we want to consider and the number of timesteps to look at for frame forecasting.
- LSTMWrapper: initiates the LSTMWrapper to be used by the LSTM model
- Train: trains the now processed data and calculates the train and test losses.
- Infer: observes the data for analysis.
- Plot: plots the data
- Analyse: analyses the performance in long-term forecast

The file 'pipeline.py' is the solution to code implementation for Tasks 2b and 2c. The shared FIFO buffer has been successfully integrated into the pipeline and can hold data used by the ML pipeline without significantly affecting its original performance.

The mutex protocols from Task 1b manage the concurrent implementation of the pipeline. Figure 1 is a diagram of the concurrent pipeline implementation. A generative AI tool was used to create the diagram in Figure 1.

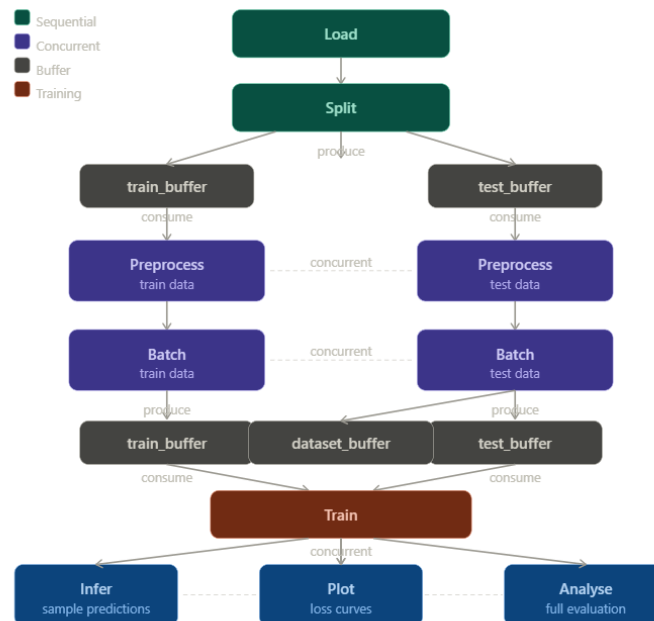
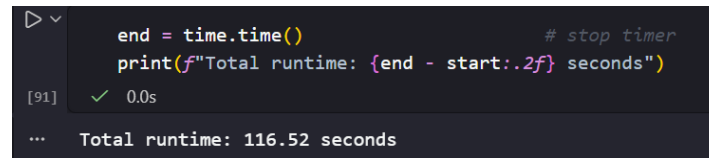


Figure 1: Pipeline of concurrent implementation

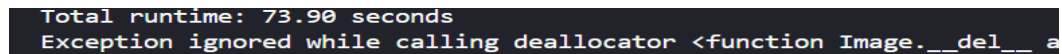
The Preprocess and Batch classes have been modified from the original code to allow the training and test data to run concurrently. The Infer, Plot and Analyse classes are also run concurrently.

Figures 2 and 3 below show the latency of the sequential ('main_script. ipynb') and concurrent('pipeline.py'), respectively and were calculated by subtracting the start time of the pipeline from the end time.



```
end = time.time() # stop timer
print(f"Total runtime: {end - start:.2f} seconds")
[91] ✓ 0.0s
... Total runtime: 116.52 seconds
```

Figure 2: Sequential Pipeline



```
Total runtime: 73.90 seconds
Exception ignored while calling deallocator <function Image.__del__ a
```

Figure 3: Concurrent Pipeline

The original sequential pipeline takes 116.52 seconds to complete, while the concurrent pipeline in Part 2 takes 73.90 seconds. The concurrent implementation saves 42.62 seconds and is 1.58x faster than the sequential.

Although the improvement is not theoretically significant, the concurrent implementation proves the importance of parallelism in embedded system designs. It is worth noting that most of the total runtime was spent during the training stage, which is currently run sequentially but can be modified to further improve performance.

Part 3 - Asynchronous Communication Mechanism (ACM) Implementation and Performance Analysis:

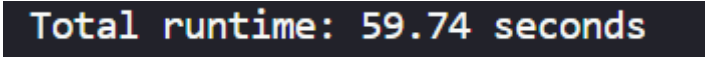
For this Task, the Pool ACM was chosen to obtain the 'most updated data' because it rereads and rewrites data in slots without waiting, writing on top of the older data, ensuring the most recent ones are used

The file 'Part_3.py' is the class implementation of a 3- slot Pool ACM. The ACM is controlled by the variables n (write slot), l (the last slot used by the writer), and r (the read slot).

The class has two functions, read and write. The 'read' function takes the last slot used to write l, reads the data in the ACM and returns it to be used by other classes in the

pipeline. The 'write' function finds a slot not being used by the reader and not the last one used and writes the data in that slot of the ACM. Data loss and data repetition are inevitable in this implementation because they occur when data are rewritten before reading or reread before overwriting, respectively. [1]

The file 'Part_3_pipeline.py' is the integration of the ACM with the classes Task 2a. It follows the same logic as Task 2b, shown in Figure 1, replacing the buffers with ACMs



Total runtime: 59.74 seconds

Figure 4: ACM Pipeline

The ACM pipeline's execution time is 59.74 seconds, as shown in Figure 4. It saves 56.78 seconds from the sequential pipeline and 14.16 seconds from the current pipeline in Part 2, yielding speed-ups of 1.95x and 1.23x, respectively.

References

- [1] A. V. Y. D. S. a. F. X. I. G. Clark, “Data Communication in Systems with Heterogeneous Timing,” November/December 2002.